CLOSED-LOOP COMMAND-RESPONSE IDENTIFICATION OF FROZEN LOCOMOTION POLICIES

A PREPRINT

Zongtan Li
zongtanli2023@gmail.com

July 2026

ABSTRACT

Learned locomotion policies are increasingly deployed as frozen, command-conditioned building blocks, driven to tasks through a velocity-command interface. We ask what can be learned about such a policy from that interface alone, without its weights, reward, or retraining, and what that knowledge is worth. We identify the frozen closed loop of policy and robot from excitation rollouts: a data-efficient model predicts, for any command in the current situation, the motion it produces, its mechanical cost, and the terrain-relative posture it leaves the robot in (displacement $R^2 \geq 0.92$, posture $R^2 \approx 0.9$, cost $R^2 \approx 0.75$ to 0.8, from about a minute of simulation; terrain-relative posture labels are essential). We use the model in two modes. As a controller, a training-free viability-gated compensator lowers the deployed policy's running cost and mechanical work across three embodiments (Go2, ANYmal-D, H1). As a measurement, it lets us bound the efficiency available from command scaling at that interface: from identical initial states, an omniscient per-episode oracle beats the best single fixed command scale by under 2%, because the efficiency-optimal scale is nearly the same in every episode. No state-conditioned controller we evaluate, ours or a heavier sampling MPC, clears that ceiling, and the ceiling persists on mixed flat–rough deployments even under a completion-time budget: terrain shifts the cost level but hardly the command that minimizes it. Efficiency compensation by command scaling therefore has a low and measurable ceiling on these locomotion policies; the contributions are the identification method that exposes it and the evaluation that measures it.

Keywords closed-loop system identification · frozen policies · command-interface control · learned dynamics models · legged locomotion

1 Introduction

Strong learned locomotion policies are increasingly deployed as frozen, command-conditioned building blocks: a task layer drives a pretrained velocity-tracking policy toward a goal through its command interface, typically with a hand-designed proportional command. In this common regime the policy's weights, reward, and training pipeline are unavailable or too costly to touch; the only thing one can freely do is *observe the policy acting* and choose what command to send next. This raises a question that is logically prior to any attempt to improve such a policy: what can be identified about a frozen policy purely from its command interface, and what does that knowledge buy?

Our first contribution is a method that answers the identification half directly. We excite the frozen policy–robot closed loop with commands sampled across its admissible range and fit a supervised model that predicts, for any candidate command in the current situation, the short-horizon motion it produces, its mechanical cost, and the terrain-relative posture it leaves the robot in. Modeling the *closed loop* at the command interface, rather than the robot's action-level dynamics as in model-based RL, keeps the input three-dimensional and makes identification a cheap supervised problem on the policy's own rollouts. The resulting model is accurate and data-efficient, and we isolate one ingredient that is essential rather than incidental: posture must be labeled in terrain-relative coordinates, without which its predictability collapses on rough ground.

The identified model is a reusable lens on the frozen policy, and we put it to work in two modes. The first is *control*. We build **Viability-Gated Command Compensation** (VGCC), a training-free controller that at each step scores a structured set of candidate commands around the proportional command and executes a bounded correction toward the predicted-cheapest candidate, but only where the model certifies that the candidate preserves task progress and terrain-relative posture, and otherwise falls back to the proportional command exactly. Across three embodiments a single configuration reduces the deployed policy's running cost and mechanical work below the proportional baseline it is layered on, without reward access, retraining, or changes to the low-level feedback.

The second mode, and the one that raises this work from a controller to a statement about the policy, is *measurement*. Because the model predicts the situational cost of every command, it lets us ask a question that logically precedes any controller: how much efficiency is there to gain at the command interface at all? We answer it against ground truth. From identical initial states we trace the cost-of-transport–time frontier of command scaling and bound it with an omniscient per-episode oracle and a same-budget sampling MPC. The finding is sharp: the frontier is *narrow*. A single tuned fixed command scale lies within 2% of an oracle that picks the best scale for every episode, because the efficiency-optimal scale barely varies from episode to episode, and no state-conditioned method we test, VGCC or a heavier MPC, clears that ceiling. This reframes a growing family of methods that improve frozen policies at the command interface: for *efficiency* on rough-terrain locomotion, the gain available from state-conditioned command scaling over a well-chosen fixed scale is bounded and small, and the identification method is precisely the instrument that measures that bound before one invests in a controller.

Contributions. (1) A method for closed-loop command-response identification of a frozen, command-conditioned policy from excitation rollouts: a data-efficient model predicting situational motion, mechanical cost, and terrain-relative posture, with terrain-relative posture labels shown to be necessary. (2) VGCC, a training-free compensator built on the identified model, that robustly improves the deployed proportional baseline across three embodiments (Go2, ANYmal-D, H1) with one configuration and an exact fallback. (3) A measurement, enabled by the identified model and validated against ground truth: through paired, iso-time, oracle, and same-budget-MPC analyses we show that the cost-of-transport frontier of command scaling is narrow: a tuned fixed command scale lies within 2% of an omniscient per-episode oracle over the same scales, a bound on what episode-level scale selection, learned or not, can buy on these policies, and one that no state-conditioned method we evaluate escapes.

2 Related Work

Feedforward and internal model control. Feedforward compensation precorrects a reference using a plant model, leaving feedback to handle residual error [Åström and Murray, 2008]; the internal model principle formalizes why regulation requires a model of the signals to be tracked or rejected [Francis and Wonham, 1976]. VGCC transports this structure to learned policies: the "plant" is the policy–robot closed loop, the model is a learned network, and inversion is a discrete comparison over a small command set. The novelty is not the control-theoretic template but its instantiation on top of a frozen RL policy, with the model identified from the policy's own rollouts.

Model-based RL. Learning a dynamics model and exploiting it for control is one of the oldest programs in RL: Gaussian process models enable data-efficient policy search [Deisenroth and Rasmussen, 2011], probabilistic neural ensembles support sampling-based planning within a handful of trials [Chua et al., 2018], short model-generated rollouts accelerate policy optimization [Janner et al., 2019], latent world models train behaviors entirely in imagination [Hafner et al., 2020], and learned models combine with terminal value functions for planning [Hansen et al., 2022]; on physical robots, neural dynamics models have driven locomotion and navigation directly [Nagabandi et al., 2018]. All of these model the environment at the *action* level so that a policy or planner can be built on top. VGCC inverts the arrangement: the policy already exists and is competent, so what is modeled is the short-horizon response of the *closed loop*, policy included, to its 3-D command input. This collapses the input dimensionality at the interface, makes inversion by enumeration trivial, and confines the decision to "commands the policy understands" rather than raw torques.

MPC in robotics and vehicles. Model predictive control optimizes a model-based cost over a receding horizon under constraints [Mayne et al., 2000], and is widely used in both vehicle and legged control: predictive steering on slippery roads [Falcone et al., 2007], sampling-based path-integral control for aggressive autonomous driving [Williams et al., 2018], learned-dynamics MPC for autonomous racing [Kabzan et al., 2019], and convex MPC for quadruped locomotion [Di Carlo et al., 2018]; and learning-based MPC with safety guarantees is an active research area in its own right [Hewing et al., 2020]. VGCC can be read as the smallest useful member of this family: a single-interval decision whose "dynamics" are an identified closed-loop response, whose decision variable is one velocity command rather than an action sequence, and whose constraint handling is a veto gate with a guaranteed fallback instead of constrained optimization. The simplification is deliberate: the low-level policy already solves the hard receding-horizon

problem at the contact timescale, so the compensator only needs to choose *which* well-understood command to issue next.

Reference governors and safety filters. The structure of VGCC has a precise control-theoretic ancestor: the reference governor, an add-on scheme that monitors and minimally modifies the reference of a prestabilized closed loop to enforce constraints [Garone et al., 2017], and its learning-era descendant, the predictive safety filter, which certifies or corrects a proposed input by model prediction [Wabersich and Zeilinger, 2021]. VGCC can be read as a *learned, cost-optimizing* governor: where governors modify the reference only as much as constraint enforcement demands, VGCC additionally optimizes a running cost at matched task progress, and its "model" is identified from rollouts rather than derived, trading the governors' invariance guarantees for applicability to policies whose dynamics exist only as a network. Two recent robot-learning systems are the most closely related. ABS [He et al., 2024] searches twist commands against a learned reach-avoid value to keep an agile quadruped policy collision-free; its objective is safety, and its value network and policy are co-trained as one framework, whereas VGCC attaches to an unmodified pretrained policy and optimizes efficiency subject to safety. Kim et al. [2024] identify linear Koopman dynamics of an RL-controlled biped, policy in the loop, for MPC-based safe navigation; VGCC likewise identifies the closed loop at the command level, but with an observation-conditioned nonlinear model whose terrain-dependent cost and posture channels have no linear lifted-state analogue, and uses it for compensation rather than navigation. To our knowledge, the specific combination that VGCC instantiates (post-hoc identification of a frozen policy's situational command response, running-cost optimization at matched predicted progress, and a learned predictive viability gate behind a bounded correction with exact fallback) has not previously been reported, although each constituent mechanism has established antecedents. More important than the controller, and unlike these prior systems, is how we use the identified model: not only to *act* on the policy but to *measure* a property of it: the state-dependent efficiency headroom its command interface exposes. Where prior closed-loop identifications are means to a control end, we treat identification as an instrument that returns a boundary result about the achievable frontier of command-level compensation, which is the contribution we most want to transfer.

Options, skills, and behavior archives. Temporal-abstraction research builds libraries of reusable behaviors: options [Sutton et al., 1999, Bacon et al., 2017], unsupervised skills [Eysenbach et al., 2019, Sharma et al., 2020], skill priors from offline data [Pertsch et al., 2020, Ajay et al., 2021], large-scale skill embeddings [Peng et al., 2022], and quality-diversity repertoires used for rapid adaptation [Mouret and Clune, 2015, Cully et al., 2015, Pugh et al., 2016]. DADS [Sharma et al., 2020] is notable here for planning over learned skill transition models, but those models are co-trained with the skills they describe; VGCC instead identifies the response of an already-trained task policy it had no part in shaping. A retrieval-based variant (Appendix B.2) reuses an online archive of behavior chunks as gated command residuals; our experiments characterize it as a partial solution that inherits the coverage limits of the archive, whereas the parametric response model interpolates over the entire command space. Hierarchical RL layers goal-conditioned controllers [Nachum et al., 2018], and recent imitation methods execute action chunks closed loop [Zhao et al., 2023]; we find raw chunk replay destabilizes contact-rich locomotion, which motivates compensation at the command interface instead.

Improving a pretrained policy without retraining it. Residual policy learning adds a learned action-space correction on top of a base controller [Silver et al., 2018, Johannink et al., 2019]; it improves the base but requires reward access and further environment interaction, and its correction shares authority at the contact-critical timescale. RMA adapts a policy online through a latent extrinsics estimate [Kumar et al., 2021], and multiplicity-of-behavior training exposes a space of gaits through the command interface [Margolis and Agrawal, 2022]; both must be built in at training time. Massively parallel pipelines [Rudin et al., 2022, Makoviychuk et al., 2021, Mittal et al., 2023, Schwarke et al., 2025] and terrain curricula [Lee et al., 2020, Miki et al., 2022] improve the policy during training. VGCC occupies the remaining cell of this matrix: it is attached after training, learns nothing by reinforcement, touches only the command interface the policy already exposes, and computes its correction by inverting an identified model. Command-level intervention preserves the base policy's low-level feedback structure, but it does not by itself guarantee balance: a poor command can still induce posture loss, which is precisely what the viability gate and bounded correction exist to prevent. That this cell is empty is itself informative. Interface-level optimization over a fixed low-level controller thrives wherever the interface exposes state-dependent structure: eco-driving optimizes speed profiles over a fixed powertrain for substantial energy savings [Sciarretta et al., 2015], and reference governors modify setpoints for safety [Garone et al., 2017]. Our boundary result (Section 4.5) suggests the absence of prior command-level *efficiency* compensation for locomotion is not an oversight but a reflection of the interface: a policy trained for robust velocity tracking exposes little efficiency structure for such a layer to exploit, and the identification this paper develops is a way to measure that before building one.

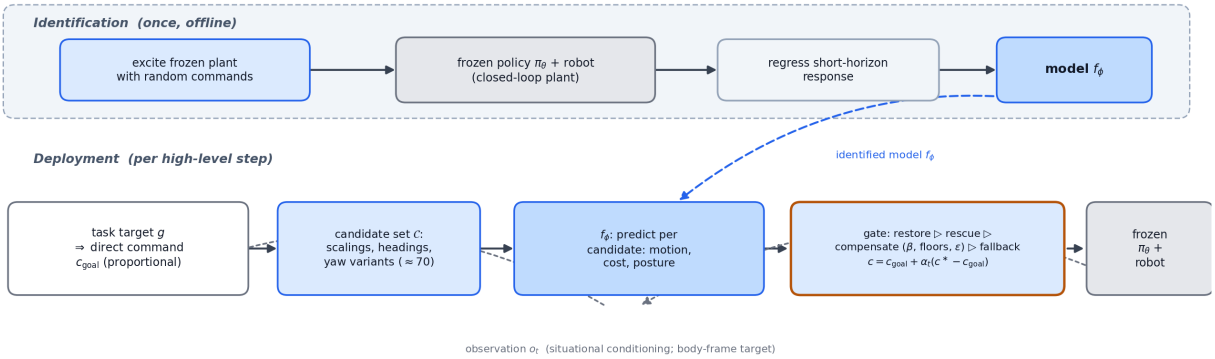

Figure 1: VGCC overview. Offline excitation rollouts from the frozen policy–robot closed loop train an observation-conditioned response model. Online, the model scores a structured candidate set around the direct command (scalings, heading offsets, yaw variants); the priority-ordered gate restores or rescues posture, compensates with a bounded annealed correction toward the cheapest eligible candidate only when the progress, posture, and cost-margin criteria pass, and otherwise uses the exact direct fallback.

3 Method

The method has two parts. The core is an offline *identification* of the frozen policy–robot closed loop from its command interface (Section 3.2): a supervised command-response model fit to the policy's own excitation rollouts. Built on it is one *application*, an online compensator (Section 3.3) that uses the model to apply, at each step, a bounded gate-certified correction toward the cheapest of a structured set of candidate commands (Figure 1). We present identification first because it is the reusable object; the compensator is the concrete use through which we test the model.

Design requirements. Three requirements shape both parts. *R1, no retraining*: everything must be obtained by only *observing the policy acting*, since its weights, reward, and training pipeline are unavailable or too expensive to touch. *R2, non-degradation*: any controller built on the model must be able to revert to the strong pretrained policy exactly, and reject its own intervention when the predicted benefit is uncertain. *R3, embodiment generality*: nothing may depend on robot-specific knowledge beyond what the identification recovers automatically from the policy's own rollouts. The architecture has a command interface (where to intervene, Section 3.1), a response model (what to know, Section 3.2), and a viability gate (how to act, Section 3.3).

3.1 Compensation Interface

Let π_θ be a frozen, pretrained continuous-control policy mapping observations o_t and a command c to low-level actions. In our locomotion instantiation $c = (v_x, v_y, \omega_z)$ is the body-frame velocity command, but the framework only requires *some* command interface through which behavior can be steered, a property shared by learned controllers deployed as task-level building blocks [Margolis and Agrawal, 2022, Kumar et al., 2021].

Scope. Three properties of the policy, not of robots, delimit where the framework applies. *(i) A command interface must exist*: a single task policy trained without conditioning exposes nothing to invert over, so VGCC does not apply to unconditioned RL policies (a limitation shared by every command-level method). *(ii) The interface must carry redundancy*: compensation exploits the existence of several commands that make comparable task progress at different cost; a task that fully determines the command admits no efficiency gain. *(iii) The closed loop must respond at a fast, roughly stationary timescale*: command consequences must be visible within a short prediction window, which holds when the policy stabilizes its plant (locomotion, end-effector servoing, vehicle velocity/curvature tracking) and fails for sparse, long-horizon credit-assignment problems. These conditions characterize command-conditioned continuous control broadly, from legged robots to vehicle controllers, rather than reinforcement learning in general; Section 5 returns to what lies outside them.

Why intervene at the command rather than at the action, as residual policy learning does [Silver et al., 2018, Johannink et al., 2019]? Three reasons, mirroring the three requirements. First, action-space corrections must be *learned*, and learning them requires reward access and environment interaction, violating R1; a command-space correction can instead be *computed* from an identified model. Second, an action-space residual shares authority with the policy at

the contact-critical timescale, whereas a command-space correction leaves π_θ in control of posture, contacts, and joints. This reduces the intervention bandwidth but does not by itself guarantee stability; R2 therefore requires explicit fallback and posture gating. Third, the command interface is the one interface that command-conditioned policies share across embodiments (R3): the same three command variables steer a quadruped and a humanoid.

A task supplies a target; throughout the method, g denotes that target expressed in the robot's body frame at the current step, and the *direct* controller computes a proportional command

$$c_{\text{goal}} \;=\; \big(\text{clip}_{[-v_x^{\max},v_x^{\max}]}(k_v x_{\text{local}}),\; \text{clip}_{[-v_y^{\max},v_y^{\max}]}(k_v y_{\text{local}}),\; \text{clip}_{[-\omega^{\max},\omega^{\max}]}(k_\psi \,\text{atan2}(y_{\text{local}}, x_{\text{local}}))\big), \quad (1)$$

which is the uncompensated baseline: a strong pretrained policy driven straight at the goal. Proportional commands of this kind are the standard way tasks are layered on velocity-conditioned policies, and they carry a systematic inefficiency: far from the goal they saturate, commanding full speed regardless of whether the terrain rewards it, and near the goal they shrink, producing energy-intensive in-place stepping. Compensation replaces c_{goal} with a corrected command at each controller decision. The design question is where the correction comes from and when it is trusted.

3.2 Command-Response Model

To decide whether a candidate command is worth executing, the compensator must predict what it would do. The object to model is not the robot's action-level dynamics, as in model-based RL [Nagabandi et al., 2018]: the policy already handles those competently, and re-modeling them would duplicate its work at far higher dimensionality. The relevant object is the *closed loop* of policy and robot, viewed from the command interface: for the current situation, it specifies the motion, cost, and postural risk that each candidate command produces. This reframing keeps the model input three-dimensional at the interface and makes identification a supervised problem on the policy's own rollouts, satisfying R1 and R3 simultaneously.

Excitation data. We override the converged policy with commands sampled throughout its admissible command space, including low-magnitude commands near the goal. This is persistent excitation applied to the policy–robot closed loop. Crucially, *no quality filtering is applied*: stumbles, slips, and low-posture episodes are kept, because the model must know where the policy performs poorly. Filtering to high-quality segments is a natural shortcut and an actively harmful one: a model trained only on clean behavior cannot warn the gate about the sloppy behavior it must avoid (Section 4.2).

Response targets. The outputs instantiate a general three-channel schema that any instantiation of the framework must fill: a *progress* statistic sufficient to score task advancement, the *cost* being minimized, and the *viability* quantity the gate guards. For locomotion, constant-command windows provide body-frame displacement $(\Delta x, \Delta y)$ and yaw change $\Delta\psi$ for progress; a mechanical cost for the objective; and minimum and mean terrain-relative posture height for viability. We use two cost channels of increasing physical fidelity. The first is a dimensionless *actuation-effort proxy* $\bar{a} = \text{mean}_{t,j}(|a_{t,j}\dot{q}_{t,j}|)$, formed from the public policy action a; because a is not a measured motor torque, this proxy has no physical energy unit and is treated as an optimization objective, not an energy measurement. The second is a torque-derived mechanical power $\bar{P} = \text{mean}_{t,j}(|\tau_{t,j}\dot{q}_{t,j}|)$, where $\tau_{t,j}$ is the simulator-applied joint torque; this is a simulated mechanical quantity, still not battery energy, but the physically meaningful channel used to audit whether reducing the proxy reduces mechanical work. A manipulation instantiation would substitute end-effector displacement, an appropriate actuation cost, and a grasp-stability or joint-limit margin. Posture height is recovered from the policy's own height-scan observation (mean scan return plus sensor offset), not from world coordinates. This prevents ground elevation from being confounded with posture; its empirical value is reported with the response-model results.

Model. The response model is a mapping

$$f_\phi : (o_t, c) \;\longmapsto\; \big(\widehat{\Delta x}, \widehat{\Delta y}, \widehat{\Delta\psi}, \hat{C}, \hat{h}_{\min}, \hat{h}_{\text{mean}}\big), \quad (2)$$

where hats mark quantities *predicted* by the model, plain symbols are measured, and $\hat{C}$ is the chosen cost channel. The implementation takes the full policy observation with its command component replaced by the candidate command; architecture, training, and split details are given in Appendix A. Conditioning on the full observation is what makes the predictions *situational*: the same candidate command is predicted to cost differently on different terrain and from different gait states, which is precisely the information a fixed command-shaping heuristic cannot express.

3.3 Viability-Gated Compensation

VGCC scores a structured candidate set built around the direct command: uniform scalings of c_{goal} (factors 0.4 to 1.2), yaw-rate offsets, and a small polar grid of absolute speeds at heading offsets around the target direction, about 70 candidates in total, all clipped to the admissible command box (exact values in Appendix A). One batched forward

pass of f_ϕ predicts every candidate's response. The set is wide enough to contain slower, redirected, and re-timed variants of the direct command, but each candidate is still a single command held for one decision interval, not a plan.

Scoring and execution. The gate scores raw candidates; what it executes is a bounded correction toward the selected candidate c^*, $c_{\text{exec}} = c_{\text{goal}} + \alpha_t\,(c^* - c_{\text{goal}})$, with gain $\alpha_t = \alpha\,\text{clip}(d/d_{\text{anneal}}, 0, 1)$ that anneals to zero as the remaining distance d shrinks, so corrections fade out on final approach. The executed command therefore always lies between the strong direct command and a model-certified candidate; scoring the raw candidate rather than the blend is a deliberate simplification whose consequence is limited by the bounded gain.

Progress floor as a slowdown bound. Predicted target progress is

$$\hat{G}(c) \;=\; \|g\| \;-\; \big\|g - (\widehat{\Delta x}(c), \widehat{\Delta y}(c))\big\|. \tag{3}$$

A candidate c is *eligible* when it keeps most of direct control's progress and leaves posture viable:

$$\hat{G}(c) \geq \beta\,\hat{G}(c_d), \qquad \hat{h}_{\min}(c) \geq h_{\text{floor}}, \qquad \hat{h}_{\text{mean}}(c) \geq h_{\text{now}} - \delta, \tag{4}$$

where $c_d = c_{\text{goal}}$ denotes the direct command. Because the prediction horizon has a fixed duration, $\hat{G}(c)$ is progress *per unit time*, so the progress floor is directly a bound on how much an accepted candidate may slow the robot down: with $\beta = 0.9$, it must advance at least 90% as fast as direct control within the horizon, capping the instantaneous slowdown at roughly 10%. This constraint is the mechanism that keeps VGCC from collapsing into the trivial "just walk slower" solution, and Section 4.4 shows empirically that removing it does exactly that.

Cost objective and margin. Among eligible candidates, VGCC selects the one with the lowest predicted horizon cost, $c^* = \arg\min_{c\,\text{eligible}} \hat{C}(c)$, and activates compensation only if the predicted saving clears a relative margin,

$$\hat{C}(c^*) < (1 - \epsilon)\,\hat{C}(c_d), \tag{5}$$

and otherwise executes c_d exactly. The division of labor between the two tests matters: predicted cost alone can always be lowered by slowing down, so the anti-slowdown guard is the progress floor in Eq. 4, which restricts the argmin to candidates that give up at most a fraction $1-\beta$ of direct control's speed, while the margin ϵ demands that what remains is a strict predicted saving rather than model noise. An intervention is admitted only when it is both no more than marginally slower *and* predicted cheaper by a margin: an efficiency gain within a bounded slowdown, not a slowdown dressed up as one. Whether that bounded slowdown nevertheless accounts for the realized savings is an empirical question, answered with slowdown-invariant metrics in Section 4.4. A seemingly more principled alternative, normalizing predicted cost by predicted progress so that the objective itself is a cost-of-transport surrogate, performs *worse* in a paired ablation (Appendix B.3): with the floor already bounding slowdown, the extra normalization double-penalizes slower candidates and suppresses most of the compensation the gate would otherwise certify.

Posture rescue and restore. Two reflexes complete the controller. If the model predicts that even the direct command will breach the posture floor within the horizon, VGCC instead selects, among candidates predicted to make non-negative progress, the one with the highest predicted minimum height (*rescue*); the bounded correction then pulls the executed command toward it. If measured posture has already sagged below the operating band, VGCC suspends compensation and issues the aggressive direct command until height recovers (*restore*). Both reflexes default toward, not away from, the strong direct controller, preserving R2. The model is thus a veto on candidate commands, not an unconstrained optimizer. All scalars are selected on development seeds and frozen before confirmation; Appendix A lists their values.

Comparing a structured single-step candidate set, rather than optimizing freely over the command box or planning over a long horizon, is a deliberate choice. The response model is reliable for the local, near-stationary consequences of a command (Section 4.2) but not for compounding predictions over a long horizon, and the margin ϵ exists precisely because ranking many similar candidates sits at the edge of its accuracy. The gated comparison uses the model only where it is trustworthy, to decide whether a cheaper certified command exists right now, and leans on the exact fallback everywhere else.

4 Experiments

The experiments follow the structure of the method. **Q1 (identification, the core method)**: how accurately and how cheaply can the closed-loop command response be identified, and what labeling makes it work (Section 4.2)? **Q2 (application)**: does a compensator built on the identified model improve the deployed proportional baseline, and does a single configuration transfer across the command space and across embodiments (Sections 4.3 and 4.6)? **Q3 (the boundary, and the paper's central finding)**: how does the compensator compare to the strongest simple alternatives (a per-robot-tuned fixed command scale, a reactive governor, and a same-budget sampling MPC), and,

Algorithm 1 Viability-Gated Command Compensation (one high-level step)

Require: frozen policy π_θ, response model f_ϕ, target g, correction gain α, anneal distance d_{anneal}, and fixed scalars $(\beta, h_{\text{floor}}, h_{\text{resc}}, \delta, \epsilon)$

Compute body-frame target g, direct command c_{goal} (Eq. 1), and the candidate set $\mathcal{C}$ around c_{goal}
Batch-predict $f_\phi(o, c)$ for all $c \in \mathcal{C}$; set $\alpha_t = \alpha \,\text{clip}(d/d_{\text{anneal}}, 0, 1)$
if measured posture is below the operating band **then**
    $c \leftarrow c_{\text{goal}}$ ▷ restore
else if direct control is predicted to breach the posture floor h_{resc} **then**
    $c^* \leftarrow \arg\max_{\mathcal{C}:\, \hat{G} \geq 0} \hat{h}_{\min}$; $c \leftarrow c_{\text{goal}} + \alpha_t(c^* - c_{\text{goal}})$ ▷ rescue
else
    $c^* \leftarrow \arg\min_{\mathcal{C}\text{ eligible (Eq. 4)}} \hat{C}$
    if $\hat{C}(c^*) < (1-\epsilon)\,\hat{C}(c_{\text{goal}})$ (Eq. 5) **then**
        $c \leftarrow c_{\text{goal}} + \alpha_t(c^* - c_{\text{goal}})$ ▷ compensate
    else
        $c \leftarrow c_{\text{goal}}$ ▷ exact direct fallback
    end if
end if
Execute π_θ with c for E low-level steps

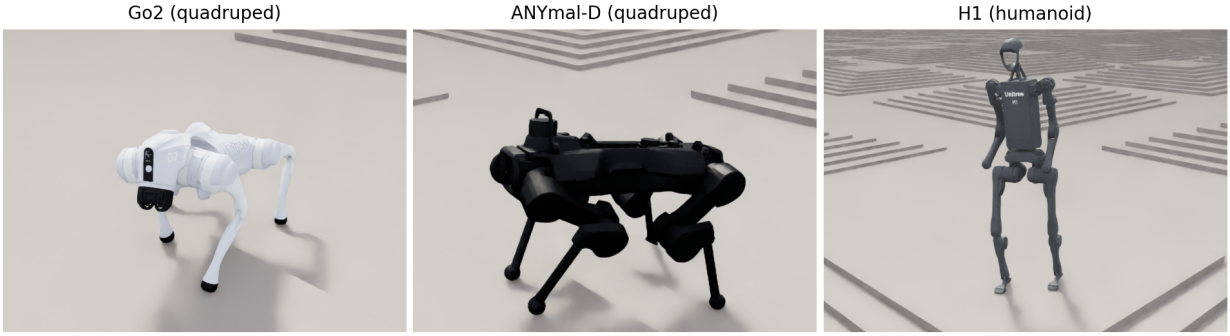

Figure 2: The three embodiments in their Isaac Lab rough-terrain velocity environments: the Go2 and ANYmal-D quadrupeds and the H1 humanoid. Each is driven by its publicly distributed pretrained RSL-RL policy, which is the policy VGCC compensates; VGCC changes only the command that policy receives, never its weights. The two quadrupeds share every gate parameter unchanged; the humanoid differs only in its posture thresholds.

underneath those comparisons, how much state-dependent efficiency structure does the command interface expose at all (Sections 4.4–4.5)? Q1 establishes the method; Q2 shows what the identified model lets us *do*; Q3 shows what it lets us *conclude*. The conclusion, that a tuned fixed scale is within 2% of an omniscient per-episode oracle, is the result we consider most important.

4.1 Setup

Environments and robots. All experiments use Isaac Lab [Mittal et al., 2023] rough-terrain velocity environments with three embodiments (Figure 2): `Isaac-Velocity-Rough-Unitree-Go2-v0` and `Isaac-Velocity-Rough-Anymal-D-v0` (quadrupeds) and `Isaac-Velocity-Rough-H1-v0` (humanoid), with the publicly distributed pretrained RSL-RL checkpoints [Rudin et al., 2022, Schwarke et al., 2025] as the compensated policies. The task is target reaching: move the base to a target g specified relative to the start pose. Success requires no early termination, final XY distance below $0.3\,\text{m}$, and *final height* above 70% of spawn height. Final height is a terminal posture check, so that an episode that reaches the target crouched, collapsed, or mid-fall does not count as a success. Because the terrain is procedurally rough, final height reflects both posture and the ground elevation where the robot stops, which is why we report it alongside, rather than inside, the cost and distance metrics.

Metrics. We report success, final distance, final height, and how often compensation was active. The optimized running cost is the dimensionless actuation-effort proxy $\text{mean}_{t,j}(|a_{t,j}\dot{q}_{t,j}|)$; since the public policy action a is not

Table 1: Go2 command-response model, held-out episode test metrics (∼25k excitation windows, 16-step horizon). Terrain-relative height labels are essential for the viability gate.

Output	MAE	RMSE	R^2
Δx (m)	0.019	0.025	0.944
Δy (m)	0.019	0.025	0.942
$\Delta\psi$ (rad)	0.029	0.042	0.927
effort proxy (per step)	0.435	0.625	0.752
min. rel. height (m)	0.009	0.012	0.875
mean rel. height (m)	0.008	0.010	0.906
min. height, world-frame labels	0.039	0.092	0.247

measured motor torque, this proxy has no physical energy unit. To test whether reducing the proxy reduces mechanical work, a torque-derived evaluation records simulator-applied joint torques τ, integrates $\sum_{t,j} |\tau_{t,j}\dot{q}_{t,j}|\Delta t$, and reports mean mechanical power, work per traveled meter, cost of transport (COT, normalized by mg and path length), and elapsed task time (Sections 4.3 and 4.4).

Baselines. (1) `direct`: the pretrained policy under the direct target command, the policy being compensated and a strong baseline; (2) `fixed scaling`: the direct command multiplied by a constant scale (0.75 or 0.90), the open-loop efficiency heuristic; (3) `reactive governor`: a measured height/tilt governor that scales the command down when measured posture degrades. Random and zero-command baselines are excluded as uninformative against a strong pretrained policy. A retrieval-based compensator (VGSR) is reported in Appendix B.2.

Task families. *Harder* Go2 targets: $(2, 0)$, $(1.5, 1)$, $(0, 1.25)$, $(-0.75, 0)$, $(1, -1)$, $(0, -1.25)$, $(-1, 0.75)$, $(1.75, -0.75)$ m: long, diagonal, lateral, and backward goals covering the command space (the last three directions were added when the benchmark was expanded and were never used in any tuning). *Moderate* Go2 targets: $(0.75, 0)$, $(0.75, 0.75)$, $(1.5, 0)$, $(0, 0.75)$ m. *Holdout* Go2 directions (never used during any tuning): $(1.25, 0.5)$, $(0.5, 1)$, $(-0.5, 0.5)$, $(1.25, -0.5)$ m. ANYmal-D and H1 use the same eight harder targets as Go2.

Protocol. VGCC's gate scalars (Section 3.3) were tuned on a *single* Go2 seed (301) on the harder set, then frozen for all Go2 and ANYmal-D evaluations. Excitation collection, identification, and every gate parameter are shared unchanged across the two quadrupeds. The humanoid retains the objective, cost margin, response-model architecture, and correction gain, but uses more conservative posture thresholds (eligibility, rescue, and restore fractions $(0.93, 0.90, 0.96)$ and height-decline tolerance 0.8 cm, versus $(0.85, 0.80, 0.90)$ and 2 cm for the quadrupeds), selected after diagnosing the humanoid failure mode in Section 4.6; H1 therefore tests morphology-adapted transfer, not zero-tuning transfer. All reported results use disjoint evaluation seeds: harder 5 seeds $\times$ 10 trials $\times$ 8 targets (400 episodes per method), moderate 3×10 (120), holdout 3×5 (60), ANYmal-D $3 \times 5 \times 8$ (120), H1 $3 \times 5 \times 8$ (120). Per-robot response models are identified with the identical recipe (Appendix A). We report episode-level bootstrap 95% CIs as descriptive uncertainty summaries. Because episodes share a small number of environment seeds, these intervals do not establish equivalence or noninferiority; paired seed-level differences are reported separately, and success comparisons are phrased as "no observed aggregate loss" rather than as proof of equality.

4.2 Identifying the Command Response

Table 1 reports held-out episode test metrics for the Go2 response model. Displacement and yaw responses are accurately predicted ($R^2 \geq 0.92$); the effort proxy is predicted well enough to rank candidates ($R^2 = 0.75$); and terrain-relative posture height is predicted reliably ($R^2 = 0.88$/0.91 for min/mean, MAE under 9 mm), versus $R^2 = 0.25$ when the same model is trained on world-frame height labels, which terrain elevation corrupts. The excitation data matters as much as the architecture: a model trained on utility-filtered archive segments (the natural "reuse the archive" shortcut) reached $R^2 = -1.9$ on lateral displacement, because curated high-quality segments underrepresent exactly the sloppy behavior the gate must anticipate. The torque-derived mechanical-power channel used for the physical evaluations is identified with the same recipe by a five-member ensemble and reaches $R^2 = 0.797$ (MAE 13.0 W).

Identification data efficiency. Re-identifying the model from episode-group subsamples degrades gracefully: with 50% of the data, displacement R^2 holds at 0.93 and the effort channel drops to 0.72; with 25% ($\approx$ one minute of parallel simulation), displacement is still 0.92 and the effort channel 0.62. Displacement, which drives the progress constraint, saturates early; accuracy on the cost channel is what additional data improves. The 25% model is evaluated in closed loop in Appendix B.3.

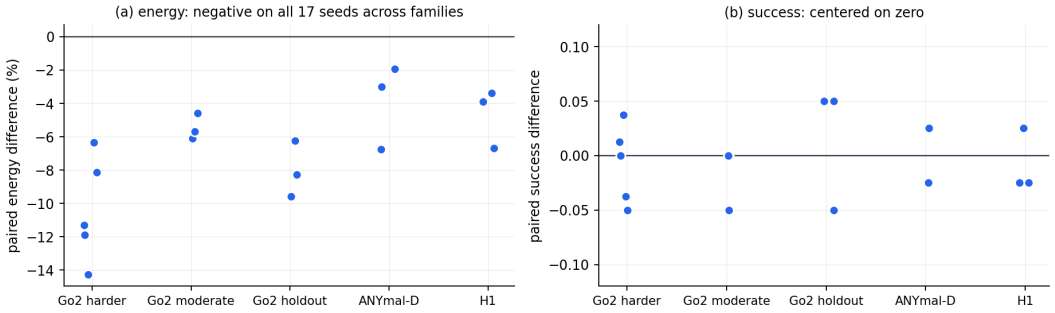

Figure 3: Paired per-seed differences (VGCC − direct) for all evaluation seeds. The effort-proxy difference is negative for 17/17 seeds (two-sided exact sign test $p = 1.53 \times 10^{-5}$). Success differences straddle zero. Points are descriptive seed aggregates; the pooled sign test combines heterogeneous families.

Table 2: Torque-derived mechanical-work evaluation of the main configuration on three new seeds ($n = 48$ episodes per method). Work is $\sum_{t,j} |\tau_{t,j}\dot{q}_{t,j}|\Delta t$ using simulator-applied torque; COT normalizes by mg and traveled path length. Values are descriptive because there are only three independent seeds.

Metric	VGCC	direct	Δ
Success	0.771	0.792	−0.021
Effort proxy	1.933	2.280	−15.2%
Absolute mechanical work (J)	129.89	141.48	−8.2%
Mean absolute power (W)	60.29	71.88	−16.1%
Elapsed task time (s)	2.113	1.905	+10.9%
Work per path length (J/m)	111.14	118.60	−6.3%
Cost of transport	0.754	0.805	−6.3%

4.3 Compensation on Go2

The harder-target benchmark comprises eight targets evaluated over 5 seeds × 10 trials, i.e. 400 episodes per method.

Running cost. VGCC reduces the actuation-effort proxy by 10.5% (1.976 vs. 2.208; descriptive episode-bootstrap CIs $[1.938, 2.016]$ vs. $[2.156, 2.256]$). Observed success is similar (0.797, CI $[0.760, 0.835]$, vs. 0.805, CI $[0.765, 0.843]$), as are final distance and height, with zero early terminations for either method. The experiment was not powered as an equivalence test, so CI overlap is not treated as evidence that success is statistically equal. The effect is directionally consistent *within* seeds: the paired per-seed difference is negative on all five benchmark seeds (−6.4% to −14.3%), and on all 17 evaluation seeds across the Go2 families, ANYmal-D, and H1 (two-sided exact sign test $p = 1.53 \times 10^{-5}$), while the paired success difference ranges from −0.05 to +0.05. Because the pooled sign test combines task families and embodiments, it supports consistency of direction, not a common effect size. Figure 3 visualizes both distributions.

Torque-derived mechanical work. To test whether the proxy reduction reflects mechanical effort, we re-evaluate the same configuration on three new seeds (2 trials × 8 targets; 48 episodes per method) while recording simulator-applied joint torque (Table 2). Absolute mechanical work decreases on all three seeds (−8.4%, −9.4%, −6.6%; −8.2% pooled). The decomposition is informative: VGCC lowers mean mechanical power by 16.1% but takes 10.9% longer, so the work saving is smaller than the power saving, and work per traveled meter and COT fall by 6.3%. Observed success differs by −2.1 points. This evaluation supports a simulated mechanical-work reduction for the main configuration; it is not an electrical or hardware energy claim, and its robustness to re-identification and to a tuned fixed scale is examined next.

4.4 Strong Baselines

A tuned fixed command scale is the simplest efficiency heuristic and a demanding baseline. We compare against it, a reactive governor, and direct control on a locked six-seed evaluation with the torque-derived power channel. Every method is evaluated from *identical* initial states (paired resets) so the comparison is within-condition, and VGCC is shown under both cost channels, the effort proxy and mechanical power, so the result does not depend on which channel is optimized (Table 3, Figure 4). We include two fixed scales: 0.90 closely matches VGCC's completion time and is the sharpest iso-time competitor, and 0.75 traces the slow, low-COT end of the frontier.

Table 3: Locked frontier comparison on six seeds ($n = 96$/method), ordered by completion time. All six methods are evaluated from *identical* initial states (paired resets), so the differences are within-condition. VGCC is shown under both cost channels. Bold marks the best value in each column. VGCC reduces work, power, and COT relative to direct control and attains the highest success, but a per-robot-tuned fixed scale traces a lower COT–time frontier: fixed 0.90 reaches a lower COT at matched time, and fixed 0.75 the lowest COT of all at a 38% time cost. A same-budget sampling MPC is discussed in the text.

Method	Success	Work (J) ↓	Power (W) ↓	Time (s) ↓	COT ↓
direct	0.792	135.07	68.09	**1.918**	0.767
fixed scaling (0.90)	0.781	126.85	57.28	2.145	0.724
VGCC (proxy cost)	**0.802**	128.20	56.99	2.207	0.737
VGCC (power cost)	**0.802**	128.28	56.67	2.220	0.737
fixed scaling (0.75)	0.781	**119.42**	**44.05**	2.648	**0.685**
reactive governor	0.771	128.51	44.69	2.947	0.740

The result both supports and bounds the paper's claim. Under the paired protocol, both VGCC channels reduce mechanical work (5.1%), power (16%), and cost of transport (3.9%) below direct control, with the COT reduction negative on all six seeds and the highest success of any method (0.802 vs. 0.792); this is consistent with Table 2. What VGCC does *not* do is dominate the fixed-scaling frontier. At essentially matched time, fixed 0.90 reaches a lower COT than VGCC (0.724 vs. 0.737, lower on five of six seeds), and fixed 0.75 the lowest COT of all (0.685) at a 38% time penalty. VGCC sits just above this frontier (Figure 4).

A more elaborate use of the model does not close the gap. A same-budget sampling MPC, which plans recursively over the scales $\{0.75, 0.90, 1.0\}$ with the identified model instead of applying VGCC's viability gate (Appendix A), was run paired on three of the six seeds. It reaches COT 0.757, below direct control (0.765) but above both VGCC (0.736) and fixed 0.90 (0.721). Planning more elaborately over the same model beats neither the fixed scale nor VGCC, for the reason Section 4.5 makes precise. We therefore do not claim VGCC beats a well-tuned fixed scale on cost of transport; its value lies instead in one configuration that needs no per-robot scale selection and transfers across the command space and embodiments (Section 4.6).

Is the gain just a slowdown? No. The metrics we headline are slowdown-invariant: cost of transport and work per meter measure energy per *distance*, so walking the same path with the same gait more slowly leaves them unchanged, and VGCC's reductions therefore require a genuinely cheaper gait. The progress floor $\beta = 0.9$ also bounds the per-step slowdown and is load-bearing: removing it deepens the proxy saving from 12.2% to 19.6% but costs 3.7 success points (Appendix B.3). The slowdown that remains is selective, so VGCC lengthens episodes by 17% where the fixed 0.75 scale pays 40%.

4.5 A Narrow Efficiency Frontier

The comparisons so far are between specific controllers. The identified model lets us ask the question underneath them, the one that determines whether a state-conditioned command-scaling controller can succeed here: how much situation-dependent efficiency structure does the policy's command interface expose? If the efficiency-optimal command scale is nearly the same in every situation, then no state-conditioned gate, however good its model, can beat a single well-chosen scale, and learned command-scaling compensation has a low ceiling. We measure the episode-level component of that ceiling directly.

For every episode we have the ground-truth cost of transport of the direct command (1.0), fixed 0.90, and fixed 0.75 from identical initial states. We compare the best single *global* scale (minimizing pooled COT) against an *oracle* that picks the best scale *per episode*. The oracle upper-bounds what any selector that holds one scale from this set for the whole episode could extract; the gap between it and the best global scale is the episode-level adaptivity headroom that exists to be won. A per-step switch (VGCC) or plan (the sampling MPC) is not formally bounded by an episode-level oracle, but neither undercuts even the best fixed scale in Table 3, let alone the oracle. Corrections outside the scale family are not searched by the oracle; VGCC's candidate set does include redirected and re-timed variants, and their availability does not change its position against the fixed scales.

That headroom is almost nothing (Figure 5): the per-episode oracle improves on the best single scale by only 1.0% on Go2 ($n = 72$ fully successful paired episodes; episode-bootstrap 95% CI $[0.5, 1.7]\%$) and 1.9% on H1 ($n = 197$; CI $[1.1, 3.0]\%$). The per-episode winners show why: the most aggressive available slowdown wins 78% of Go2 episodes and 74% of H1 episodes, and COT falls monotonically as the scale drops, so the efficiency-optimal command is a near-constant global slowdown, not an episode-dependent choice. This single measurement explains the entire pattern

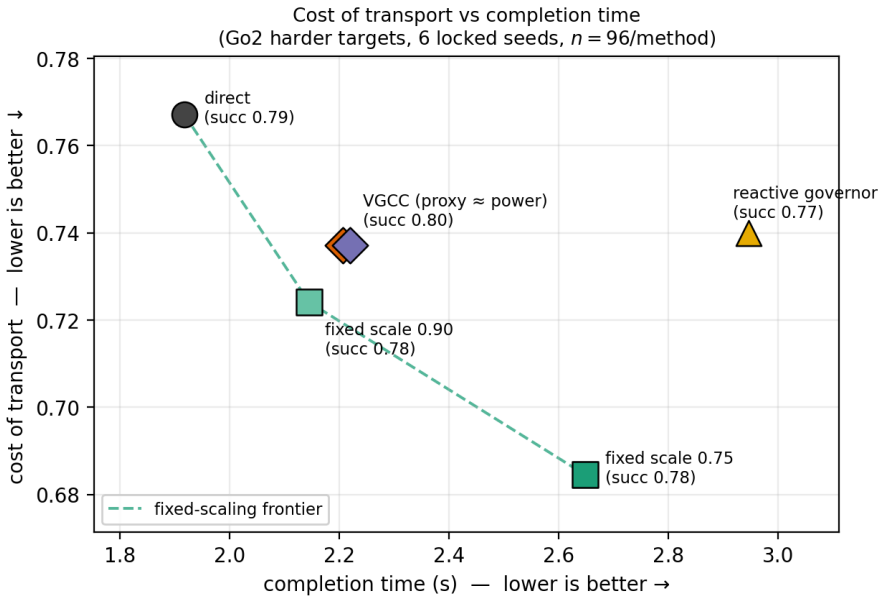

Figure 4: Cost of transport versus completion time on six locked seeds (Go2 harder targets, $n = 96$/method, identical initial states). The dashed line is the frontier traced by the direct policy and two fixed command scales. VGCC reduces COT relative to direct control but sits just above the fixed-scaling frontier, while attaining the highest success (0.80). The reactive governor is dominated. VGCC's advantage is not frontier dominance but that a single configuration reaches this near-frontier operating point with no pre-selected scale, keeps the highest success, and (Section 4.6) transfers across the command space and across embodiments, where a fixed scale must be chosen per robot and per deadline in advance. The narrow vertical spread of all methods anticipates Section 4.5: even an omniscient per-episode oracle barely improves on the fixed scale (Figure 5), so this band is close to all that command scaling can buy.

of results in this section: VGCC lands just above the fixed scale, the sampling MPC lands even further off, and the reactive governor is dominated, not because any of them models the policy poorly, but because the situational structure they exist to exploit is nearly absent. There is a structural reason to expect exactly this: the policy was trained to track commands robustly across terrains and states, and robustness *is* insensitivity, so a well-trained tracking policy actively suppresses the state-dependence a command-level efficiency selector would need. The interface exposes little structure because the training objective worked. The identification method that a practitioner would use to *attempt* the compensation is thus also the cheapest instrument for discovering, in advance, that the compensation is not worth deploying: on this policy class, a well-chosen fixed command scale is within 2% of the per-episode optimum, and command-scaling efficiency compensation is a solved, low-ceiling problem.

Does terrain heterogeneity open the frontier? A natural conjecture is that mixed terrain would: if rough ground rewards slowing more than flat ground does, a terrain-conditioned scale should beat any single fixed one across a mixed deployment. We test this directly with two single-terrain-type variants of the same environment, a flat plane and a uniformly rough field (2–10 cm height noise), the same pretrained policy and protocol, a response model re-identified on the mixture with the unchanged recipe, and the scale grid extended to 0.60 (three fresh seeds, eight targets, two trials, paired resets; 48 episodes per scale per terrain). Because the world-frame final-height criterion is dominated by stop-point ground elevation on ± 10 cm noise, completion here means reaching the target without early termination. Table 4 shows the outcome: cost of transport falls monotonically with the scale on *both* terrains, so the efficiency-optimal scale is the bottom of the grid everywhere and the winner never flips. Terrain shifts the *level* of the cost curve but hardly its *shape* over commands, which is the mechanism above made visible. Coupling the two terrains with a shared completion-time budget, so that a deadline forces choosing *where* to slow down, changes little: allocating scales by terrain (slower on rough, faster on flat) beats the best budget-feasible uniform scale by at most 1.6% of total work (bootstrap 95% CI $[0.8, 8.4]$) at the tightest budgets, and by nothing once the budget permits slowing everywhere. The re-identified VGCC configuration behaves as on the main benchmark, tracking but not undercutting the best fixed scale on either terrain. The conjectured mechanism does not open the frontier on the roughness axis, with or without a deadline.

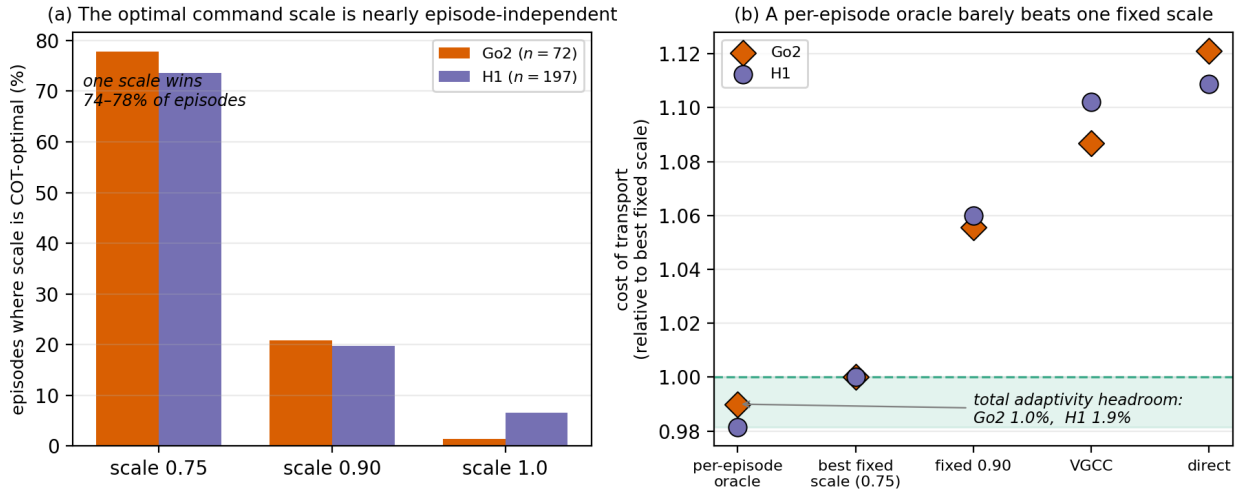

Figure 5: The paper's central finding, on paired-successful episodes (identical initial states). **(a)** The COT-optimal command scale is nearly the same in every episode: a single scale is optimal in 74–78% of episodes on both robots, so there is little for a scale selector to specialize to. **(b)** Cost of transport relative to the best single fixed scale ($= 1$): an *omniscient* per-episode oracle that picks the best scale for every episode lies only 1.0% (Go2) and 1.9% (H1) below that fixed scale; the entire shaded band is the episode-level adaptivity headroom available to scale selection. VGCC and direct control sit above the fixed scale, not because they model the policy poorly but because the headroom they compete for is nearly empty.

Table 4: Single-terrain-type paired evaluation (Go2, three seeds, eight targets $\times$ two trials; completion = target reached without early termination; flat $n = 48$, rough $n = 36$ jointly completed episodes per scale, of 48 evaluated). COT falls monotonically with the scale on both terrains: roughness raises the cost level but barely moves where its minimum lies over commands, so no fixed-scale winner flips between terrains.

	Flat			Rough (2–10 cm)		
Scale	COT	Work (J)	Time (s)	COT	Work (J)	Time (s)
1.00	1.027	179.6	1.80	1.670	290.5	2.16
0.90	0.964	166.7	1.99	1.598	278.5	2.30
0.75	0.897	152.6	2.36	1.513	265.2	2.76
0.60	0.882	148.8	2.90	1.447	247.8	3.31

We are careful about scope. This bound is measured on rough-terrain reaching with these three policies. It is specific to what the oracle searches: episode-constant scales from the grid a deployment would sweep. Both the best fixed scale and the oracle move together with a denser or lower grid, so the headline figures are grid-specific, and within-episode schedules, which an episode-level oracle does not bound, are probed only at development scale (Section 5). Terrain axes we did not vary, slopes, compliance, or friction, could in principle reshape rather than merely shift the cost curve, and interfaces richer than a velocity scaling, such as gait parameters [Margolis and Agrawal, 2022], are exactly where the tracking objective has not flattened the structure. The contribution here is not that adaptivity can never help, but that its headroom is measurable, and on the standard benchmark, and on the mixed-terrain deployments built from it, that headroom is small.

4.6 Coverage and Transfer

A compensation layer should improve efficiency where the base policy is wasteful and avoid degrading it where the base policy is strong, using one configuration. Table 5 evaluates the frozen configuration on the remaining Go2 target families and on two further robots, testing whether the single closed-loop configuration improves on direct control everywhere without per-family or per-robot objective tuning.

Table 5: Coverage across task families and embodiments. Δ is the relative change in the actuation-effort proxy. Success CIs are descriptive, not equivalence tests. Go2 and ANYmal-D use the frozen quadruped configuration; H1 uses morphology-adapted posture thresholds (Section 4.1).

Family (n/method)	Method	Success ↑	Effort ↓	Effort Δ	Final height (m)
Go2 moderate (120)	VGCC	0.975 [0.942, 1.000]	**1.743** [1.689, 1.802]	-5.4%	0.356
	direct	**0.992** [0.975, 1.000]	1.844 [1.763, 1.932]	N/A	**0.362**
Go2 holdout (60)	VGCC	**0.933** [0.867, 0.983]	**1.936** [1.836, 2.037]	-8.0%	**0.334**
	direct	0.917 [0.833, 0.983]	2.105 [1.982, 2.234]	N/A	0.333
ANYmal-D, 8 targets (120)	VGCC	**0.883** [0.825, 0.942]	**1.367** [1.317, 1.422]	-3.9%	0.500
	direct	0.875 [0.817, 0.933]	1.422 [1.373, 1.471]	N/A	0.500
H1, 8 targets (120)	VGCC	0.967 [0.933, 0.992]	**0.690** [0.665, 0.716]	-4.7%	0.928
	direct	**0.975** [0.942, 1.000]	0.724 [0.696, 0.752]	N/A	**0.941**

Moderate and holdout targets. The effort proxy drops on both families (-5.4% and -8.0%), while observed success differs by -1.7 and $+1.6$ percentage points. The holdout family is the most informative case: these directions were never observed during any tuning, so the frozen gate generalizes across the command space, a property the retrieval variant does not possess (Appendix B.2: the archive variant *increases* the proxy on these directions).

Third embodiment, zero re-tuning: ANYmal-D. Applying the recipe unchanged to a different quadruped reduces the effort proxy by 3.9%; observed success is 0.883 vs. 0.875, with identical final posture and zero terminations. The paired effort difference is negative on all three ANYmal seeds. Since no gate or objective parameter was adjusted, this is consistent with a cross-quadruped effect rather than a Go2-only artifact. A single robot and three seeds are nevertheless insufficient to establish broad embodiment generality.

H1 humanoid: the one morphology-dependent component is the posture floor. Transferring VGCC to the humanoid is not zero-shot. With the quadruped posture thresholds applied unchanged, VGCC's own model-selected aggressive commands drive the tall, high-inertia H1 into posture collapse on the long forward and diagonal targets, costing success (the 2 m target falls to 0.47): the loose quadruped floors permit commands that a much higher center of mass and smaller support region cannot sustain, so the failure is loss of balance, not a cost misprediction. The remedy is morphological, not a re-tuning of the objective: tightening the eligibility, rescue, and restore posture thresholds while leaving every non-posture parameter, including the cost margin, identical to the quadruped configuration. With the floors adapted, VGCC reduces the H1 effort proxy by 4.7% with success 0.967 vs. 0.975 and the paired effort difference negative on all three seeds (-3.4%, -3.9%, -6.7%). Because the thresholds were changed after observing the failure, this row is morphology-adapted evidence, not zero-shot transfer, and needs confirmation on fresh humanoid tasks. The posture floor is thus the single component that depends on morphology; the objective, cost margin, model architecture, and correction gain are shared unchanged across all three robots.

4.7 Ablations

A paired component study (Appendix B.3, $n = 80$ per variant) isolates the role of each mechanism. Removing the posture gate forfeits most of the proxy saving (-4.8% vs. -12.2%) and reduces success; removing the progress floor deepens the saving to -19.6% but reduces success by 3.7 points; and full command substitution ($\alpha = 1$) has the largest success cost (-5.0 points), which is why the correction is bounded. The gate keeps predicted savings inside the model's well-behaved posture regime, the progress floor limits behavioral drift, and bounded blending avoids the success cost of substitution. Quarter-data identification retains displacement $R^2 = 0.92$ but halves the realized proxy reduction, tracking the cost-channel degradation in Section 4.2.

5 Limitations and Future Work

Energy validity. The large-sample benchmark optimizes and reports a dimensionless actuation-effort proxy, not energy. Torque-derived evaluations show lower simulated mechanical work, power, and COT than direct control, both for the main configuration (Table 2) and in the paired locked comparison on every seed (Table 3). What is *not* demonstrated is an advantage over the fixed-scaling frontier: a per-robot-tuned fixed scale reaches a lower COT than VGCC, marginally at matched time and substantially at a large time cost (Figure 4). The consistent effect is therefore reduced power and running cost with a modest, adaptive work saving over the deployed policy, not a demonstrated advantage in mechanical energy over a tuned fixed scale. Absolute joint work also counts negative work without a

motor-efficiency model and is not electrical consumption. Hardware voltage/current measurements and actuator losses remain necessary for a real energy claim.

Completion time. Every efficiency the compensator realizes is entangled with completion time: the progress floor bounds the per-step slowdown, but the aggregate episode still runs 10–14% longer, and the current objective penalizes time only implicitly through that floor. An explicit completion-time budget would make VGCC provably iso-time, so that any residual saving is unambiguously a gait-efficiency gain rather than a slowdown; folding time into the ranking itself is not the fix, as the objective ablation (Appendix B.3) shows it over-suppresses compensation. The mixed-terrain analysis of Section 4.5 measures the headroom that survives a shared time budget at 1.6% of total work (CI $[0.8, 8.4]$) on the flat–rough mixture, so the deployable question is not whether budgeted headroom exists but whether any predictor can capture the little there is, on deployments richer than the roughness axis.

Statistical validity. Episode-level bootstrap intervals treat episodes as exchangeable even though they are nested within a small number of seeds. We expose seed-level paired differences to make this dependence visible, but the present seed counts are too small for precise family-level uncertainty, and overlapping success intervals do not demonstrate equivalence. A confirmatory study should pre-register a noninferiority margin for success and use a hierarchical or cluster bootstrap over seeds and targets, with roughly ten independent seeds per family.

The efficiency boundary is fundamental here, not incidental. The fixed-scaling and reactive-governor baselines show that the interesting comparison is against the speed–energy frontier, and VGCC does not dominate it. The oracle analysis (Section 4.5) makes clear this is not merely a modeling shortfall to be closed with a better response model: on uniform rough terrain the best command scale is nearly episode-independent, so even a perfect per-episode scale selector gains under 2% over a single fixed scale. A learned state-conditioned gate can only beat a fixed scale where the efficiency-optimal command genuinely varies with state. Section 4.5 closes the most natural version of that conjecture: on flat–rough mixtures, with or without a completion-time budget, the optimal scale barely varies and terrain allocation buys at most 1.6%. What remains open is where the structure has not been flattened by the tracking objective: command interfaces richer than a velocity scaling, such as gait parameters [Margolis and Agrawal, 2022]; terrain axes that reshape rather than shift the cost curve, such as slopes or compliance; and hardware, where actuator losses add state dependence that simulation lacks. Until such structure is demonstrated, a well-tuned fixed scale remains the stronger baseline on cost of transport. The identification method (Section 4.2), by contrast, stands independently of this boundary.

Scope. All results are in simulation, on target reaching, with velocity-commanded locomotion policies across three embodiments. The framework's assumptions (Section 3.1) characterize command-conditioned continuous control, not reinforcement learning in general: an unconditioned single-task policy exposes no interface to invert over, a task without command redundancy leaves nothing to save, and sparse long-horizon problems hide command consequences from any short response model. Within those bounds, the untested breadth is real: end-effector-commanded manipulation and style-commanded character control [Peng et al., 2022] fill the progress/cost/viability schema of Section 3.2 naturally and are the most direct test of generality. The recipe requires per-robot identification (a few minutes of simulation and training per robot); within the quadruped pair, identification is the only per-robot component, while H1 additionally adapts posture settings. Hardware transfer, where true energy savings matter most, is the natural next step, and would require handling safe excitation and model shift explicitly.

Model horizon and inversion. The 16-step response horizon captures immediate consequences but not slow drift; the restore reflex compensates using measured height feedback, and a multiple-horizon or recurrent model could subsume it. Inversion by comparing a small structured candidate set is cheap and robust but coarse; gradient-based inversion through f_ϕ, or a learned amortized compensator, could refine it at some cost to the robustness that enumeration provides against model error. Richer tasks (path following, disturbance recovery, velocity schedules) would exercise the progress constraint differently and remain untested.

6 Conclusion

We asked what can be identified about a frozen, command-conditioned locomotion policy from its command interface alone. Our answer is a method that identifies the policy–robot closed-loop command response from excitation rollouts, cheaply and accurately: displacement and yaw are predicted at $R^2 \geq 0.92$, terrain-relative posture at $R^2 \approx 0.9$, and a mechanical-cost channel at $R^2 \approx 0.75$–0.8, from about a minute of parallel simulation, with terrain-relative posture labels the one ingredient that is necessary rather than incidental. We then used this identified model as a lens on the policy in two modes. As a *controller*, VGCC, a training-free compensator that, behind a learned predictive posture gate, applies a bounded correction toward the predicted-cheapest of a structured candidate set, robustly reduces the deployed policy's running cost and mechanical work below the proportional baseline across three embodiments, with an exact fallback and no retraining. As a *measurement*, the same model turns a vague design question into a number:

from identical initial states we bound the cost-of-transport frontier of command scaling with a per-episode oracle and a same-budget MPC, and find it narrow: a tuned fixed command scale is within 2% of an omniscient per-episode selector, because the efficiency-optimal scale is nearly the same in every episode.

This is the paper's central result, and it is a boundary as much as a method. It says that for *efficiency* on rough-terrain locomotion, command-level schemes that rescale a frozen policy's commands are chasing a small prize: over a well-chosen fixed scale, the achievable episode-level gain is bounded, and neither our gate nor a heavier planner clears it. The value of the identification is that it makes this verdict cheap and predictive: a practitioner can fit the model in a minute of simulation and read off, before building any controller, whether the command interface carries enough state-dependent structure to be worth a learned one. We tested the most natural opening of the frontier, mixed flat and rough terrain under a completion-time budget, and found it closed: terrain moves the cost level but hardly the command that minimizes it. Where structure may still live is in interfaces the tracking objective does not flatten, gait parameters rather than a velocity scaling, on terrain that reshapes the cost curve, and on hardware with true energy measurement; establishing whether a learned state-conditioned gate can strictly dominate a fixed scale there is the experiment this study most directly motivates.

Reproducibility

All experiments use public Isaac Lab environments and public pretrained RSL-RL checkpoints. Excitation collection, response-model training, evaluation, and plotting code, together with all result files aggregated in this paper, are available in the project repository.

References

Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.

Bruce A. Francis and W. Murray Wonham. The internal model principle of control theory. *Automatica*, 12(5):457–465, 1976.

Marc Peter Deisenroth and Carl Edward Rasmussen. PILCO: A model-based and data-efficient approach to policy search. In *International Conference on Machine Learning (ICML)*, pages 465–472, 2011.

Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018. arXiv:1805.12114.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. arXiv:1906.08253.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1912.01603.

Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. In *International Conference on Machine Learning (ICML)*, 2022. arXiv:2203.04955.

Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, and Sergey Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 7559–7566, 2018. arXiv:1708.02596.

David Q. Mayne, James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

Paolo Falcone, Francesco Borrelli, Jahan Asgari, Hongtei Eric Tseng, and Davor Hrovat. Predictive active steering control for autonomous vehicle systems. *IEEE Transactions on Control Systems Technology*, 15(3):566–580, 2007.

Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou. Information-theoretic model predictive control: Theory and applications to autonomous driving. *IEEE Transactions on Robotics*, 34(6): 1603–1622, 2018.

Juraj Kabzan, Lukas Hewing, Alexander Liniger, and Melanie N. Zeilinger. Learning-based model predictive control for autonomous racing. *IEEE Robotics and Automation Letters*, 4(4):3363–3370, 2019.

Jared Di Carlo, Patrick M. Wensing, Benjamin Katz, Gerardo Bledt, and Sangbae Kim. Dynamic locomotion in the MIT Cheetah 3 through convex model-predictive control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–9, 2018.

Lukas Hewing, Kim P. Wabersich, Marcel Menner, and Melanie N. Zeilinger. Learning-based model predictive control: Toward safe learning in control. *Annual Review of Control, Robotics, and Autonomous Systems*, 3:269–296, 2020.

Emanuele Garone, Stefano Di Cairano, and Ilya Kolmanovsky. Reference and command governors for systems with constraints: A survey on theory and applications. *Automatica*, 75:306–328, 2017.

Kim P. Wabersich and Melanie N. Zeilinger. A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129:109597, 2021.

Tairan He, Chong Zhang, Wenli Xiao, Guanqi He, Changliu Liu, and Guanya Shi. Agile but safe: Learning collision-free high-speed legged locomotion. In *Robotics: Science and Systems (RSS)*, 2024. arXiv:2401.17583.

Jeonghwan Kim, Yunhai Han, Harish Ravichandar, and Sehoon Ha. Learning Koopman dynamics for safe legged locomotion with reinforcement learning-based controller. *arXiv preprint arXiv:2409.14736*, 2024.

Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2):181–211, 1999.

Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.

Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1802.06070.

Archit Sharma, Shixiang Gu, Sergey Levine, Vikash Kumar, and Karol Hausman. Dynamics-aware unsupervised discovery of skills. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1907.01657.

Karl Pertsch, Youngwoon Lee, and Joseph J. Lim. Accelerating reinforcement learning with learned skill priors. In *Conference on Robot Learning (CoRL)*, 2020. arXiv:2010.11944.

Anurag Ajay, Aviral Kumar, Pulkit Agrawal, Sergey Levine, and Ofir Nachum. OPAL: Offline primitive discovery for accelerating offline reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.13611.

Xue Bin Peng, Yunrong Guo, Lina Halper, Sergey Levine, and Sanja Fidler. ASE: Large-scale reusable adversarial skill embeddings for physically simulated characters. In *ACM Transactions on Graphics (SIGGRAPH)*, volume 41, 2022. arXiv:2205.01906.

Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.

Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. Robots that can adapt like animals. *Nature*, 521(7553):503–507, 2015.

Justin K. Pugh, Lisa B. Soros, and Kenneth O. Stanley. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 3:40, 2016.

Ofir Nachum, Shixiang Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023. arXiv:2304.13705.

Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018.

Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6023–6029, 2019.

Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA: Rapid motor adaptation for legged robots. In *Robotics: Science and Systems (RSS)*, 2021. arXiv:2107.04034.

Gabriel B. Margolis and Pulkit Agrawal. Walk these ways: Tuning robot control for generalization with multiplicity of behavior. In *Conference on Robot Learning (CoRL)*, 2022. arXiv:2212.03238.

Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning (CoRL)*, pages 91–100, 2022. arXiv:2109.11978.

Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac Gym: High performance GPU based physics simulation for robot learning. In *NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2108.10470.

Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023. arXiv:2301.04195.

Clemens Schwarke, Mayank Mittal, Nikita Rudin, David Hoeller, and Marco Hutter. RSL-RL: A learning library for robotics research. *arXiv preprint arXiv:2509.10771*, 2025.

Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47):eabc5986, 2020.

Takahiro Miki, Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62):eabk2822, 2022.

Antonio Sciarretta, Giovanni De Nunzio, and Luis Leon Ojeda. Optimal ecodriving control: Energy-efficient driving of road vehicles as an optimal control problem. *IEEE Control Systems Magazine*, 35(5):71–90, 2015.

A Implementation Details

All values below are the ones used for the paper's experiments; the exact launch commands are in the project repository.

Excitation and dataset. Excitation overrides the converged policy's command with commands drawn uniformly over the admissible box $[-1, 1]^3$ in (v_x, v_y, ω_z), resampled every 25 low-level steps; with probability 0.3 the drawn command is scaled by 0.3, oversampling the low-magnitude regime that proportional control visits near the goal. Collection runs 64 parallel environments (8,000 steps per environment for H1). Constant-command windows of 16 low-level steps (0.32 s at the 50 Hz control rate) are sliced at stride 4 within episodes, giving the ≈25k Go2 windows of Table 1. Terrain-relative height labels are recovered from the policy's own height-scan observation (mean scan return plus its 0.5 m sensor offset).

Response model and training. f_ϕ is a multilayer perceptron with two hidden layers of width 256 (ReLU, dropout 0.1), mapping the full policy observation, with the command slice replaced by the candidate command, to the six response targets; inputs and targets are standardized. Training uses AdamW (learning rate 10^{-3}, weight decay 10^{-4}), batch size 1,024, 200 epochs, keeping the checkpoint with the best held-out loss. The held-out set is 20% of episode groups, so no episode straddles the split; Table 1 reports this held-out set. The torque-derived mechanical-power channel is identified with the same recipe by a five-member ensemble.

Candidate set. The structured set $\mathcal{C}$ around the direct command comprises: uniform scalings $\{0.4, 0.6, 0.8, 1.2\}\, c_{\text{goal}}$ and c_{goal} itself; yaw-rate offsets of $\{\pm 0.2, \pm 0.4\}$ rad/s applied to c_{goal}; and a polar grid of absolute speeds $\{0.25, 0.5, 0.75, 1.0\}$ m/s at heading offsets $\{0, \pm 0.3, \pm 0.6\}$ rad around the target bearing with yaw-rate scalings $\{0, 0.5, 1.0\}$ of the direct yaw command. All candidates are clipped to the admissible box $[-1, 1]^3$ and deduplicated, leaving ≈70.

Controller constants. One high-level decision is taken every $E = 4$ low-level steps. The gate scalars shared by all three robots are: correction gain $\alpha = 0.5$ with anneal distance $d_{\text{anneal}} = 0.5$ m (the correction fades out linearly inside 0.5 m of the target), progress floor $\beta = 0.9$, and cost margin $\epsilon = 0.1$; the cost channel $\hat{C}$ is the effort proxy for the main configuration and predicted mechanical power for the power configuration of Table 3. The morphology-dependent posture floors are the eligibility, rescue, and restore fractions of nominal height with the height-decline tolerance δ: $(0.85, 0.80, 0.90)$ with $\delta = 2$ cm for both quadrupeds, $(0.93, 0.90, 0.96)$ with $\delta = 0.8$ cm for H1 (Section 4.1).

Sampling MPC baseline. The same-budget MPC of Section 4.4 plans over per-step scale sequences drawn from $\{0.75, 0.90, 1.00\}$ with planning depth 4 high-level steps, rolling the identified models forward, and executes the first command of the best sequence; it shares the response model, decision rate, and command bounds with VGCC.

Mixed-terrain evaluation (Section 4.5). The flat and rough single-type tasks replace the terrain generator of the otherwise unchanged rough environment with a single sub-terrain each: a plane, and uniform height noise of 2–10 cm; every scene, observation, command, and event setting is identical, and the same pretrained checkpoint is used. The response model for these deployments is identified once on pooled excitation from both terrains with the unchanged recipe (≈25k windows; displacement $R^2 = 0.92$, cost 0.73). Fixed scales $\{0.60, 0.75, 0.90, 1.00\}$ and VGCC run from identical initial states on three seeds disjoint from all tuning. The budget analysis pairs one flat with one rough leg, sweeps a shared completion-time budget over the feasible range, and compares the best budget-feasible uniform scale against the best per-terrain scale allocation on the convex hulls of the per-terrain (work, time) means; the reported interval is an episode bootstrap over both legs.

B Secondary Experiments and Diagnostics

This appendix retains material that is useful for diagnosis and reproducibility but is not required for the main claim: qualitative and per-target results, a retrieval-based alternative, and the complete component ablation.

B.1 Qualitative and Per-Target Results

The selected example forecasts a posture-floor breach and activates rescue, but the aggregate residual failure mode is low-posture arrival. A symmetric posture-aware stopping rule did not improve success at $n = 400$ per method; it instead spent the remaining budget attempting recovery. Individual examples must therefore not be interpreted as a success-rate effect; the aggregate evidence is in Tables 2 and 5.

Table 6 gives the per-target breakdown of the harder benchmark. The proxy reduction holds on seven of eight targets, but success differences change sign at $n = 50$ per target, which is why we report aggregate success rather than a target-specific success effect.

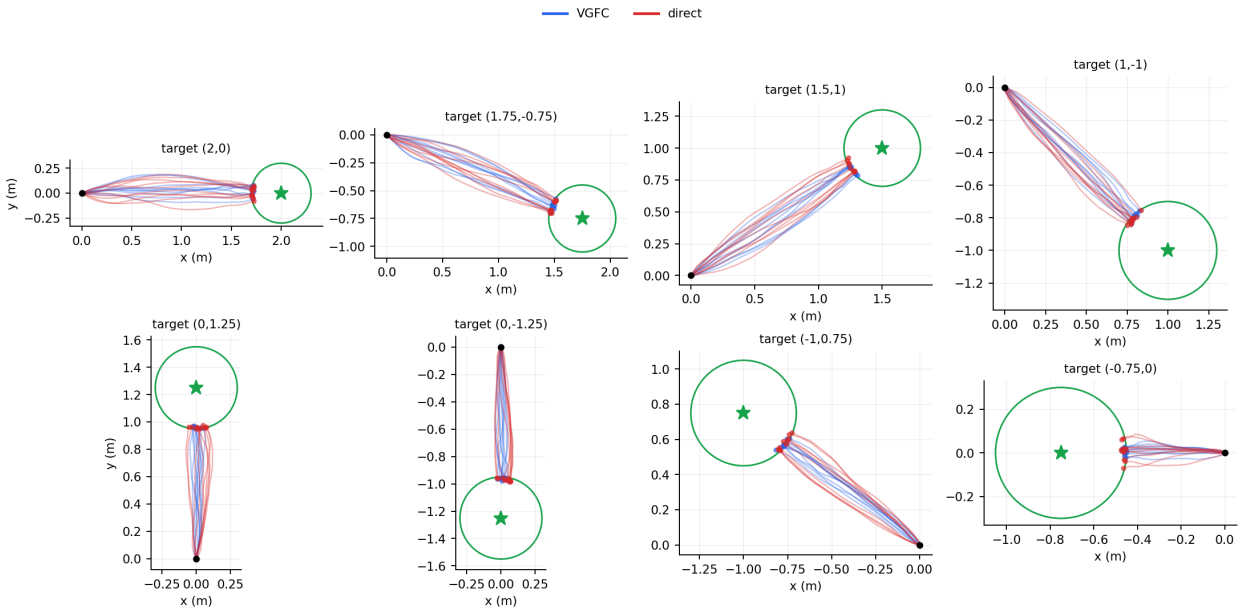

Figure 6: XY paths on the eight harder targets (one evaluation seed, 10 trials per method). VGCC and direct control reach similar target neighborhoods.

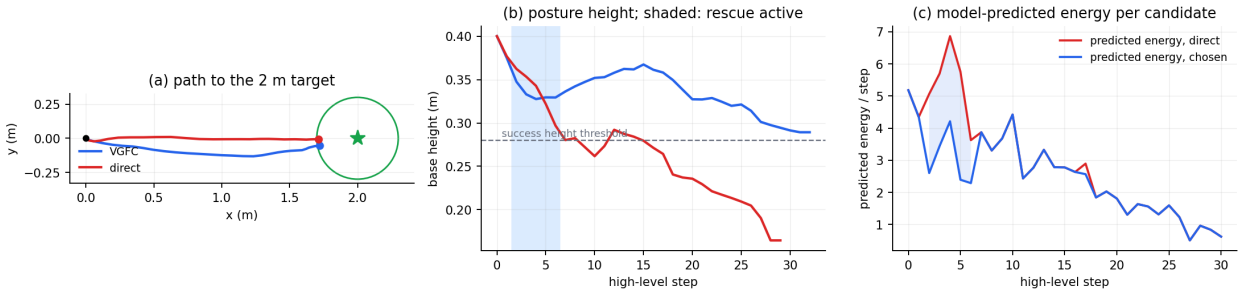

Figure 7: Selected paired episode illustrating rescue behavior. This example shows mechanism, not prevalence.

B.2 Retrieval Alternative (VGSR)

An earlier, nonparametric alternative to the response model retrieves reusable behavior chunks instead of predicting command responses. VGSR segments converged-policy rollouts into chunks, describes them by body-frame displacement, velocity, yaw, posture, effort proxy, stability, and smoothness, then novelty-filters and clusters them into a behavior archive. At deployment it retrieves a chunk by progress, state applicability, effort, stability, and utility, and blends the chunk velocity with the direct command only when a heuristic gate accepts it. Unlike VGCC, it can interpolate only inside archive support.

On the original five-target protocol, VGSR matches VGCC's proxy level on the harder family (2.061 versus 2.073) but has 1.2% terminations and lower final height. It *increases* the proxy by 3.8% on held-out directions and 4.4% on H1, exactly the generalization the parametric model provides and the archive does not. In a separate five-seed harder-target aggregate it reduces the proxy by 7.8% with success 0.785 versus 0.800. Removing its gate increases the proxy by more than 50%. Archive size is non-monotonic: the top 2,000 chunks improve success relative to all 3,904, while top-500 minimizes proxy at lower success. These coverage and curation dependencies motivate the parametric response model used by VGCC.

Table 6: Per-target harder-target results (50 episodes per target and method), ordered by proxy reduction.

	Effort proxy ↓			Success ↑		Final height (m)	
Target	VGCC	direct	Δ	VGCC	direct	VGCC	direct
$(0, 1.25)$	1.815	2.083	-12.9%	0.940	0.980	0.340	0.351
$(1.75, -0.75)$	2.217	2.542	-12.8%	0.480	0.520	0.276	0.285
$(-1, 0.75)$	1.861	2.122	-12.3%	1.000	1.000	0.358	0.367
$(0, -1.25)$	1.822	2.073	-12.1%	0.940	0.980	0.345	0.350
$(1.5, 1)$	2.203	2.471	-10.8%	0.800	0.740	0.316	0.313
$(2, 0)$	2.367	2.619	-9.6%	0.240	0.260	0.233	0.224
$(1, -1)$	2.028	2.240	-9.5%	0.980	0.960	0.355	0.349
$(-0.75, 0)$	1.496	1.510	-0.9%	1.000	1.000	0.359	0.363

Table 7: Paired component ablations on two seeds ($n = 80$ per variant). Differences are against direct control in the same runs.

Variant	Δ proxy ↓	Δ success	Active steps
VGCC (full)	-12.2%	$+0.012$	11.7
no annealing	-12.2%	$+0.013$	12.3
no viability gate	-4.8%	-0.025	8.9
no progress floor	-19.6%	-0.037	19.8
substitution ($\alpha = 1$)	-14.0%	-0.050	16.3
25% identification data	-6.4%	-0.012	10.7

B.3 Complete VGCC Component Ablation

The gate keeps predicted savings inside the model's well-behaved posture regime; the progress floor limits behavioral drift; and bounded blending avoids the larger success cost of substitution. A quarter of the identification data retains displacement $R^2 = 0.92$ but reduces the cost-channel R^2 from 0.75 to 0.62, halving the realized proxy reduction. An absolute proxy margin fails across embodiments (it increases the H1 proxy by 13.5%), so the relative margin in Eq. 5 is retained.

Objective ablation. A variant that ranks candidates by predicted cost *per predicted progress* (with execution-aligned scoring, all other mechanisms unchanged) was evaluated paired against the standard gate on the locked frontier protocol (three seeds, $n = 48$ episodes per method, identical initial states). It forfeits most of the compensation: effort proxy 2.010 versus 1.810 for the raw-cost gate (direct 2.132), mechanical work 132.3 versus 126.9 J, success 0.792 versus 0.854, and it intervenes less (elapsed time 2.08 versus 2.25 s). The per-seed paired proxy difference favors the raw-cost gate on all three seeds. With the progress floor already capping slowdown, dividing by predicted progress double-penalizes slower candidates, so the variant rejects interventions the floor would have certified; the anti-slowdown role belongs to the constraint, not the objective.